CS-107 Lecture 15
Operating Systems

Troy Astarte

Megan Venn-Wycherley

Recap: assembly language

- **Assembly language** uses mnemonic codes in place of binary instructions, one mnemonic per machine instruction.
- **The assembler translates it,** turning mnemonics, labels and pseudo-ops into the machine code the processor executes.
- **Labels name memory locations,** so a program refers to five rather than to an address the programmer has to remember.
- **Branching writes to the program counter,** which is how conditionals and loops are built out of the fetch-decode-execute cycle.

Recap: the Little Man Computer

- **The Little Man Computer** is a simulator with an accumulator, a program counter, numbered memory locations, and an in-box and out-box for I/O.
- **Eleven instructions cover the lot:** INP, OUT, HLT, STA, LDA, DAT, ADD, SUB, BRA, BRZ and BRP.
- **Each instruction is a three-digit code** stored in the same numbered locations as the data, so program and data share one memory.
- **The programs we wrote** read and echoed a value, stored two values and printed them back, and added or subtracted with a branch on the result.

Goals for Today

- What an operating system is, and where it sits between hardware and applications
- The services it provides, and how it manages resources
- How operating systems evolved: serial processing, batch, multiprogramming, timesharing
- Processes, and the states they move between
- Queues, suspension and swapping to disk
- The process control block, and what happens during a context switch

Role of an Operating System

Software Categories

- **Application software**

Software written to address specific needs: to solve problems in the real world

- **System software**

Software that manages a computer system at a fundamental level

Roles of an Operating System

- System software that **manages computer resources**, such as memory and input/output devices
- provides an **interface** through which a human can **interact** with the computer
- allows an application program to interact with these other **system resources**

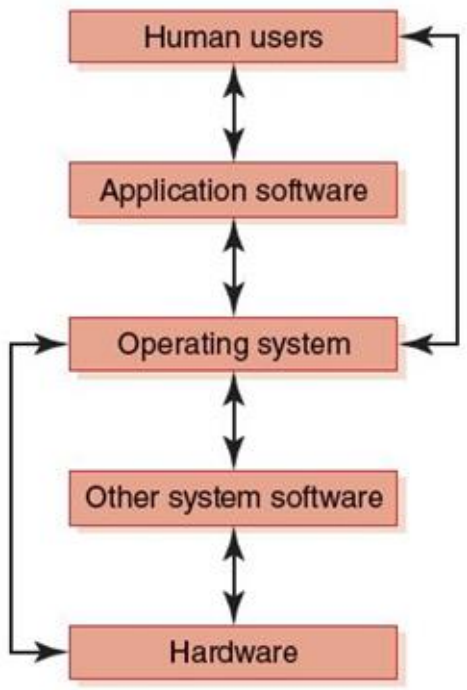


FIGURE 10.1 An operating system interacts with many aspects of a computer system.

Roles of an Operating System

- The various roles of an operating system generally revolve around the idea of "sharing nicely"
- An operating system manages resources, and these resources are often shared in one way or another among programs that want to use them

Services Provided by the OS

- **Program development**
  - Editors and debuggers.
- **Program execution**
  - OS handles scheduling of numerous tasks required to execute a program
- **System access**
  - Controls access to the system and its resources

Services Provided by the OS

- **Controlled access to files**
  - Accessing different media but presenting a common interface to users
  - Provides protection in multi-access systems
- **Access I/O devices**
  - Each device will have unique interface
  - OS presents standard interface to users
  - Device drivers – A small program that "knows" the way a particular device expects to receive and deliver information

Services Provided by the OS

- **Error detection and response**
  - Internal and external hardware errors
  - Software errors
  - Operating system cannot grant request of application
- **Accounting**
  - Collect usage statistics
  - Monitor performance

Resource Management

Resource Management

- **Multiprogramming**

The technique of keeping multiple programs that compete for access to the CPU in main memory at the same time so that they can execute

- **Memory management**

The process of keeping track of what programs are in memory and where in memory they reside

Resource Management

- **Process**

A program in execution

- **Process management**

The act of carefully tracking the progress of a process and all of its intermediate states

- **CPU scheduling**

Determining which process in memory is executed by the CPU at any given point

Evolution of Operating Systems

Evolution of Operating Systems

- Serial Processing
- Simple Batch Systems
- Multi-programmed Batch Systems
- Time Sharing Systems

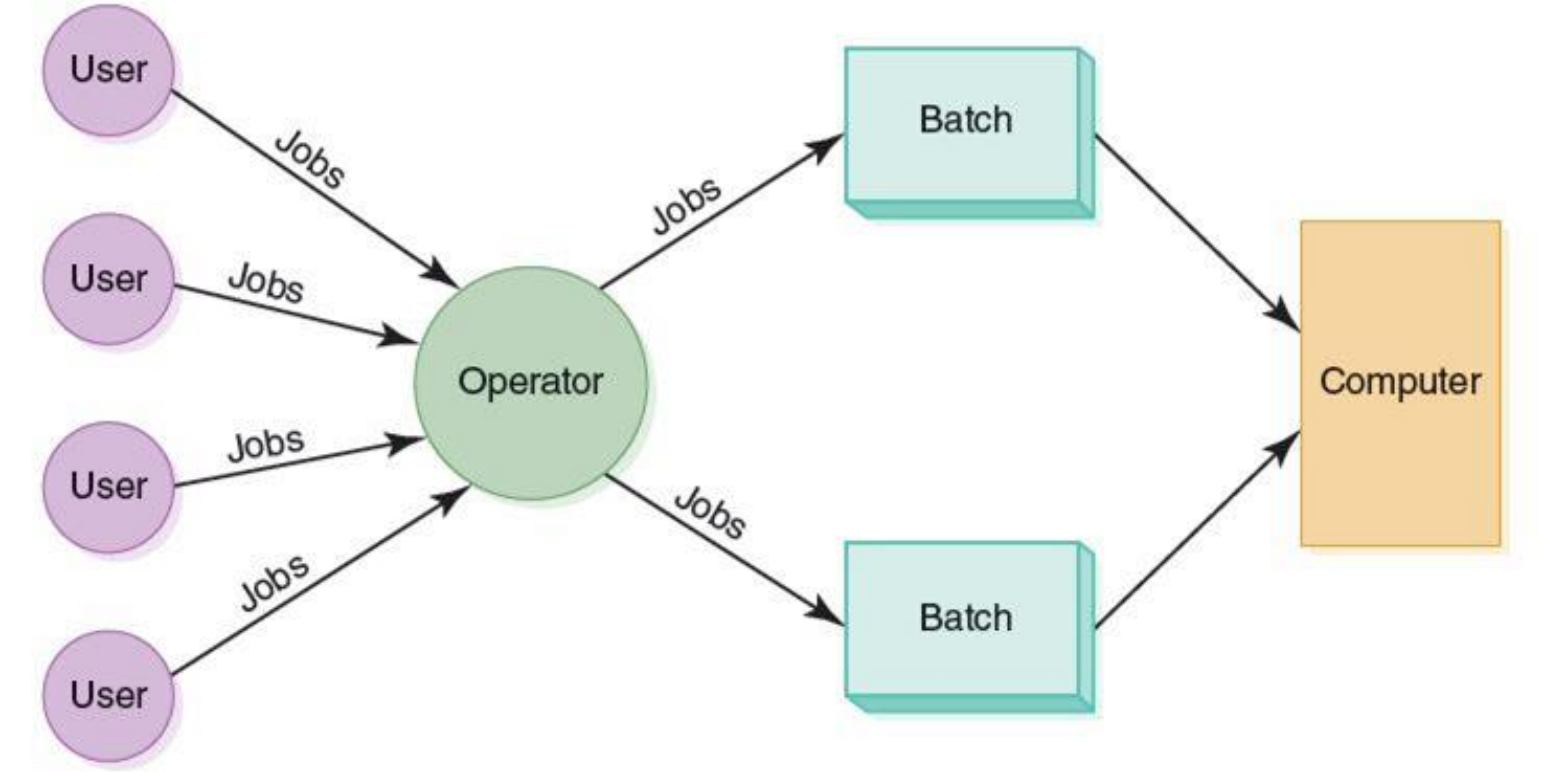
Evolution of Operating Systems

- **Serial Processing**
  - No operating system
  - Machines run from a console with display lights, toggle switches, input device, and printer

- **Problems include:**
  - Scheduling
  - Setup time

Batch Processing

- First operating system was a human operator, organizing various jobs in



Desirable Features

- **Memory Protection for OS**
  - Jobs cannot overwrite or alter OS
- **Timer**
  - Prevents a job monopolizing the system
- **Privileged Instructions and Interupts**
  - Should be able to prioritise certain actions
  - Should be able to stop/pause others

Batch Processing

Even in Batch processing, CPU is often idle.

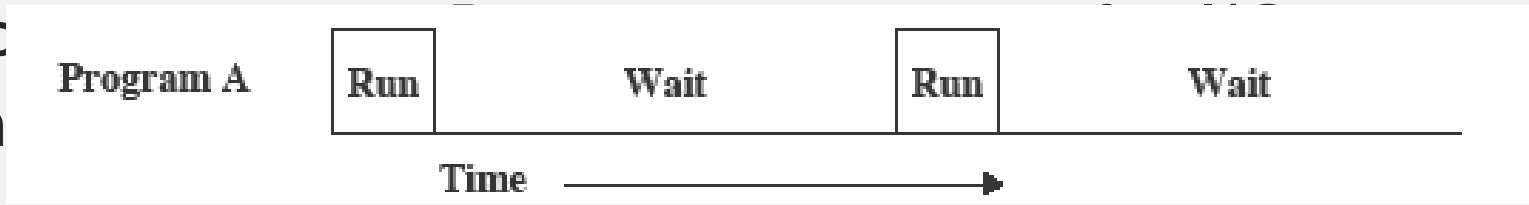
I/O devices a n memory.

Read one record from file	15 μs
Execute 100 instructions	1 μs
Write one record to file	15 μs
TOTAL	31 μs

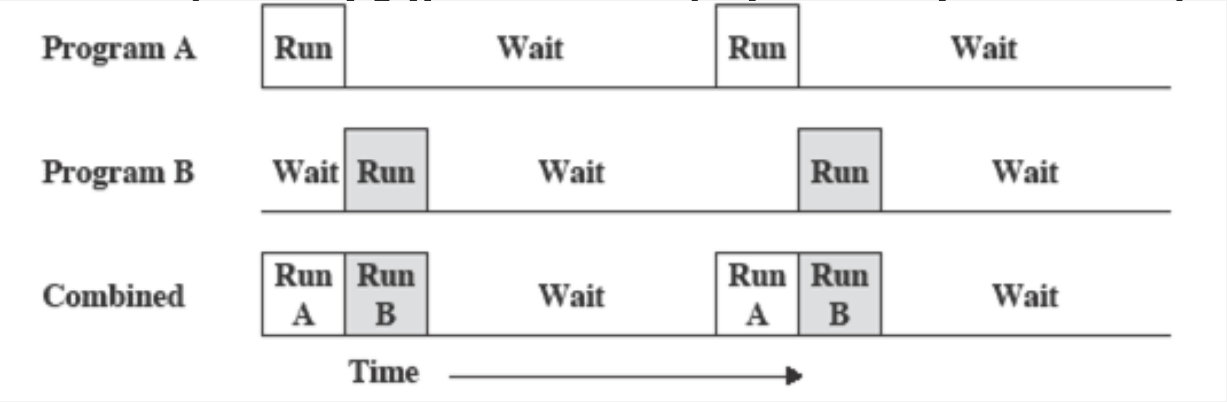
$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

Batch Processing Approaches

- Unip tion to com



- Multiprog for I/O, the processo



Timesharing

- **Timesharing system**

A system that allows multiple users to interact with a computer at the same time

- **Virtual machine**

The illusion created by a time-sharing system that each user has his/her own machine

Other Factors

- **Real-time System**

A system in which response time is crucial given the nature of the application

- **Response time**

The time delay between receiving a stimulus and producing a response

- **Device driver**

A small program that "knows" the way a particular device expects to receive and deliver information

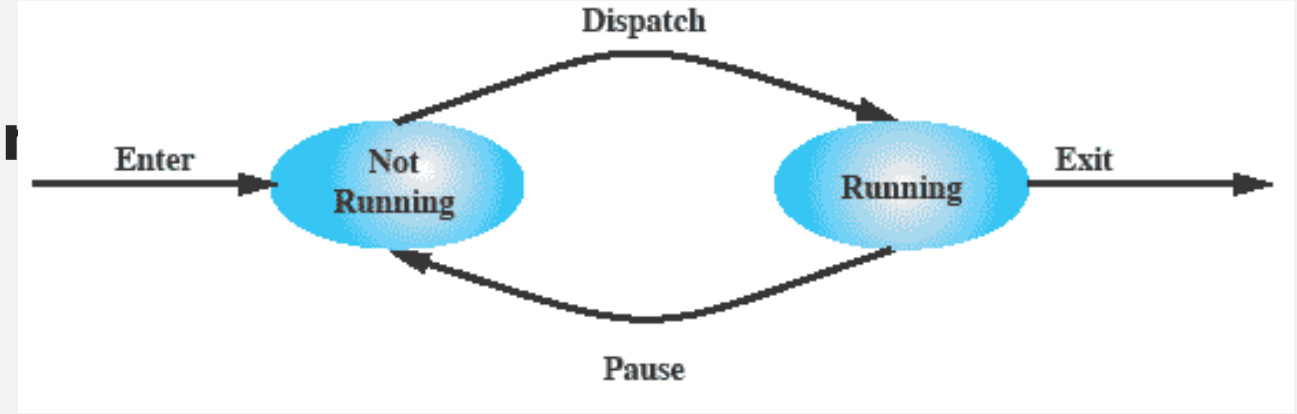
Process Management

Process Management

The act of managing the use of the CPU by individual processes

In the simplest model, a process may be in one of two **states**:

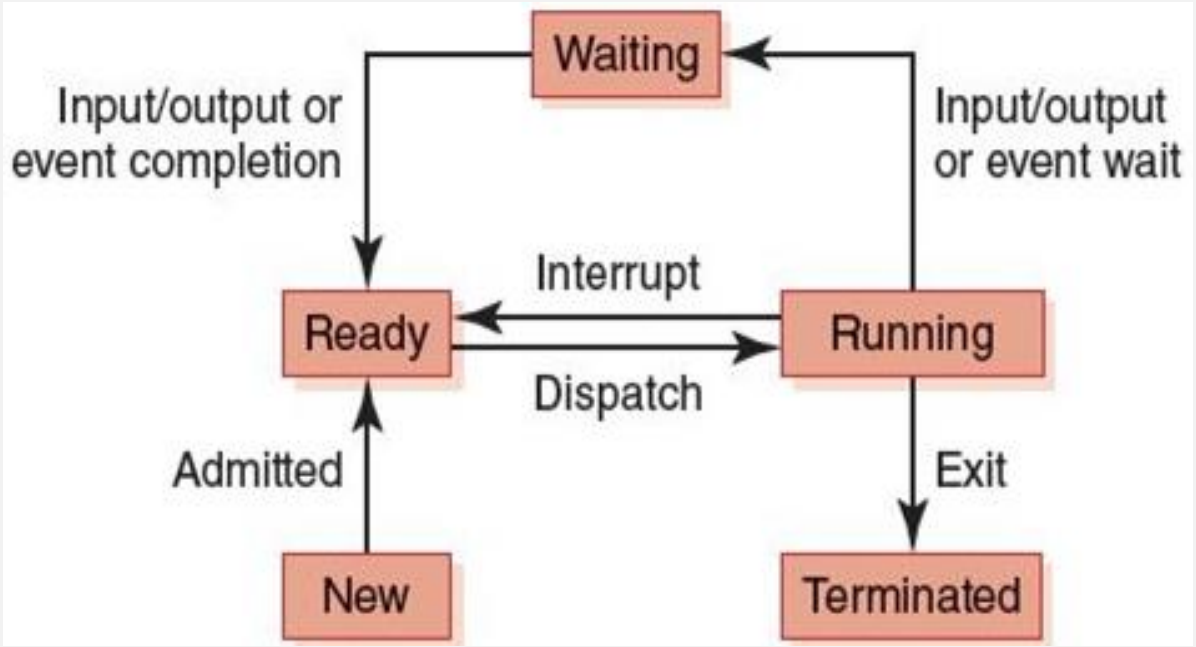
- **Running**
- **Not Runn**



Process Management

In a more complex model we can introduce states for other scenarios:

- **New**
- **Ready**
- **Waiting**
- **Running**
- **Terminated**



Process Creation

Traditionally, the OS created all processes.

But it can be useful to let a running process create another..

Parent Process - The original, creating, process

Child Process - The newly created process

Process Termination

- Various reasons to terminate a process:
  - New batch job
  - HALT/STOP instruction
  - Interactive Login
  - User logs off
  - Service / Application completed
  - Fault / Error
  - Terminating a Parent Process can terminate child processes
  - Operating System or User interrupt
  - …

Process Suspension

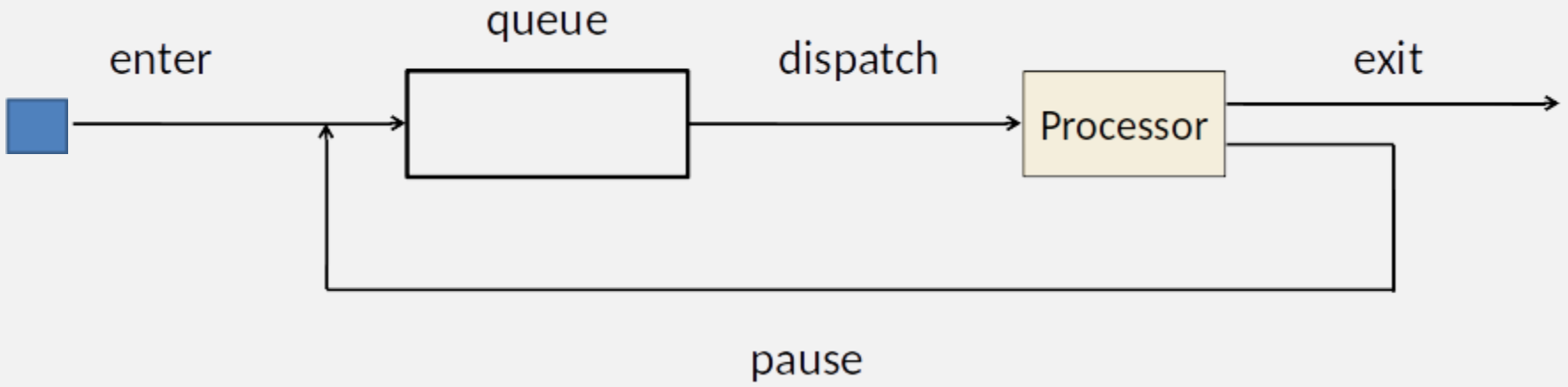
- Simple batch processes may run to completion, but to improve CPU utilisation and interactive response times we may need to suspend processes.

Processes may be suspended because:

- Process needs an unavailable resource
- OS suspects the process of causing a problem
- Interactive user wishes to pause process (debugging etc)
- Process may only need to run periodically
- OS needs resources for another process
- …

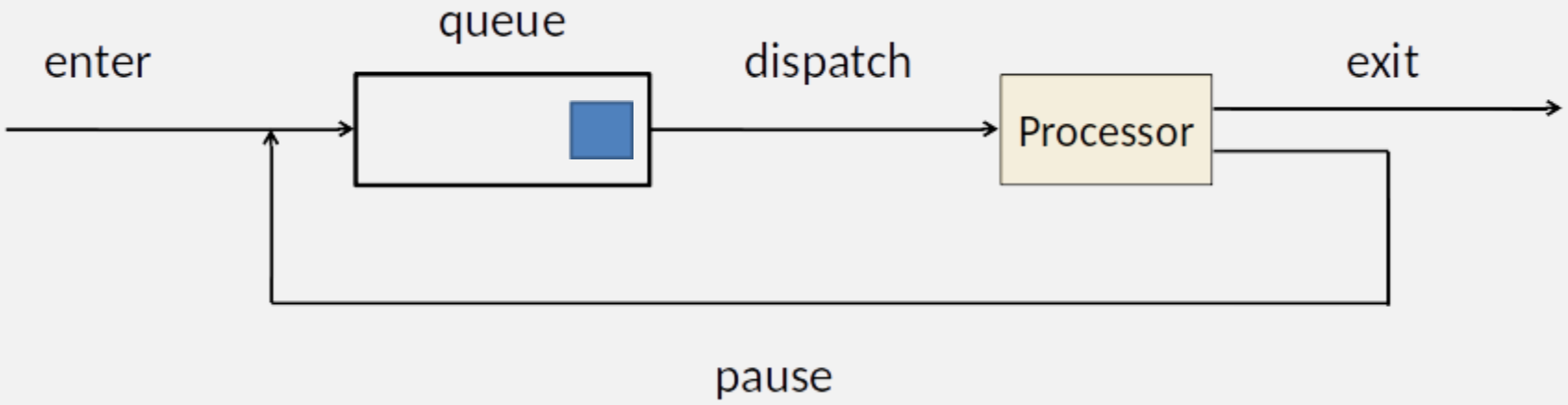
Process Management

- To manage the process, we could use a simple queue:



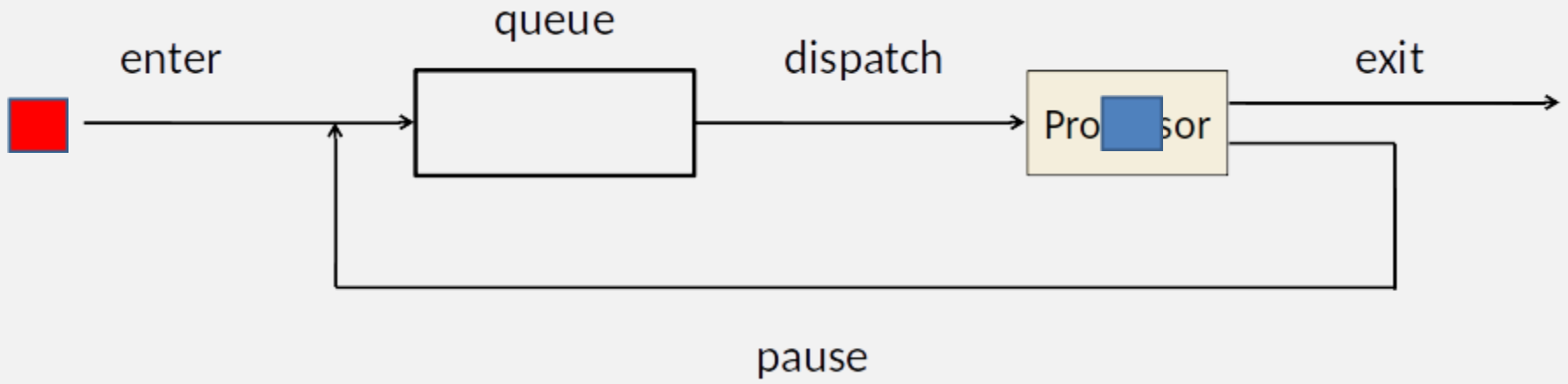
Process Management

- To manage the process, we could use a simple queue:



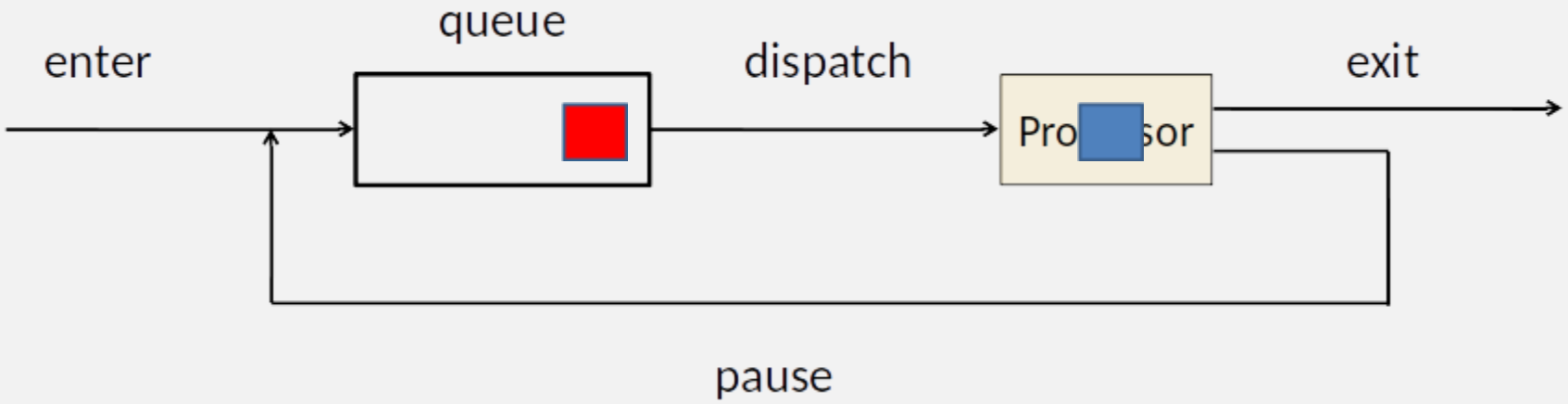
Process Management

- To manage the process, we could use a simple queue:



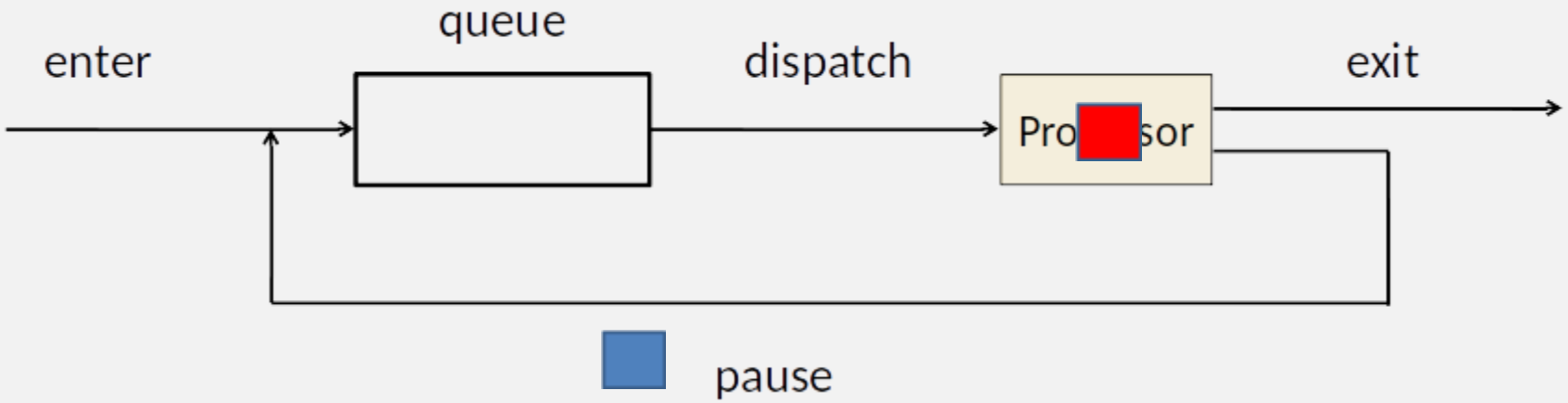
Process Management

- To manage the process, we could use a simple queue:



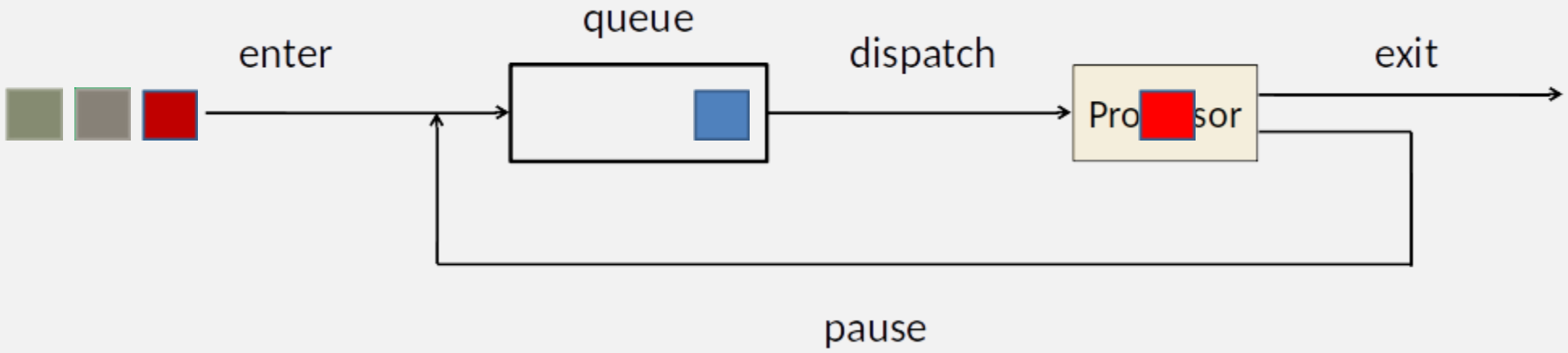
Process Management

- To manage the process, we could use a simple queue:



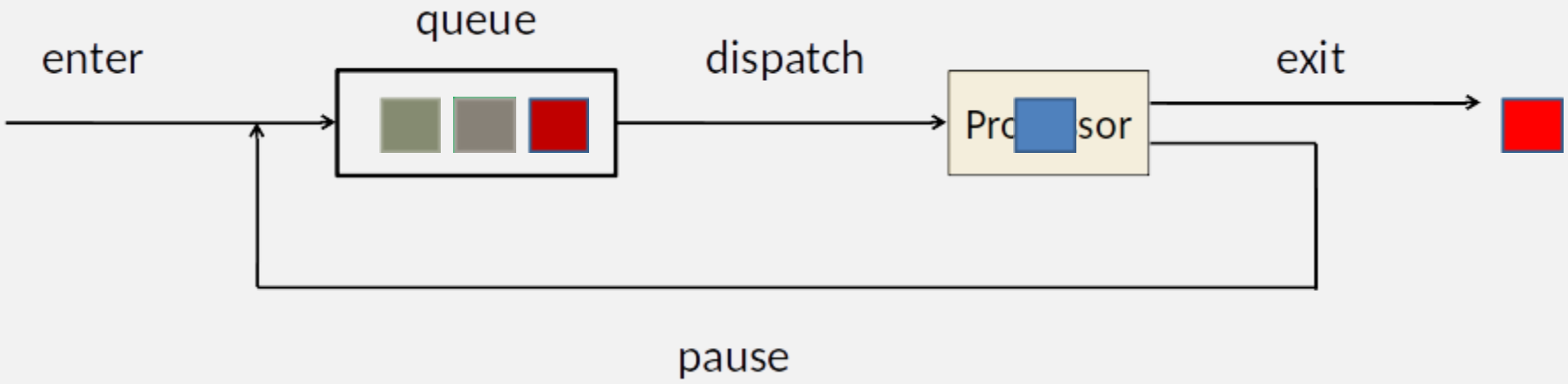
Process Management

- To manage the process, we could use a simple queue:



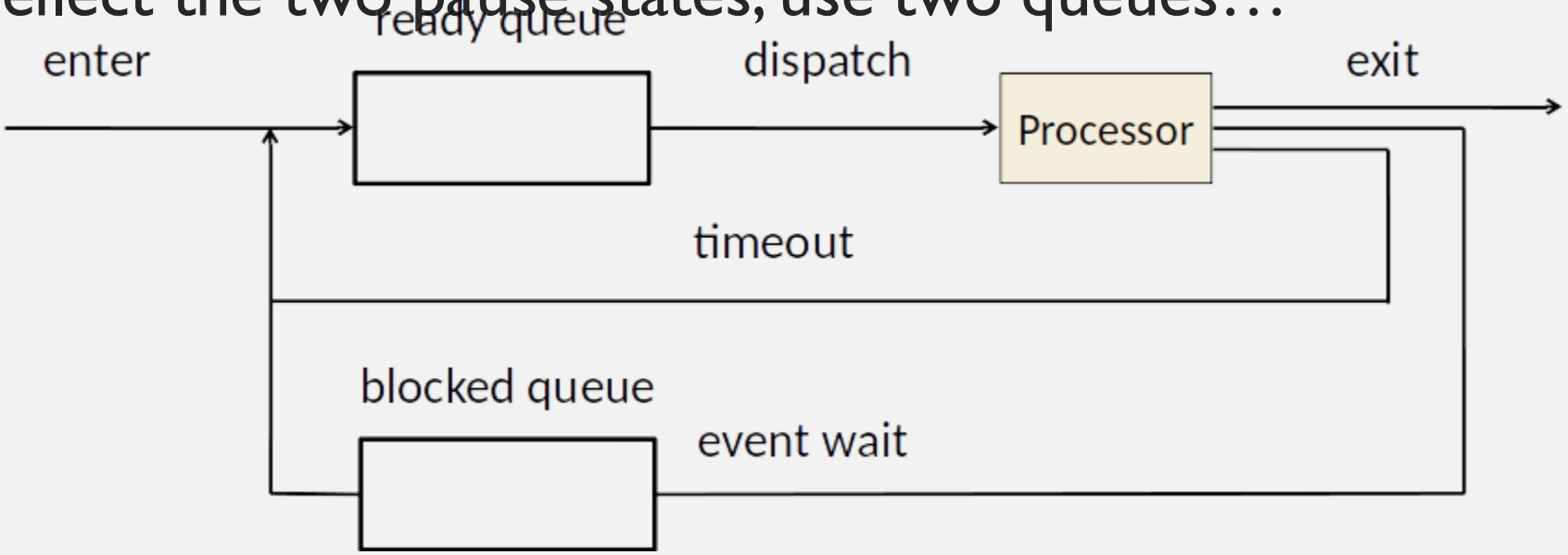
Process Management

- To manage the process, we could use a simple queue:



Process Management

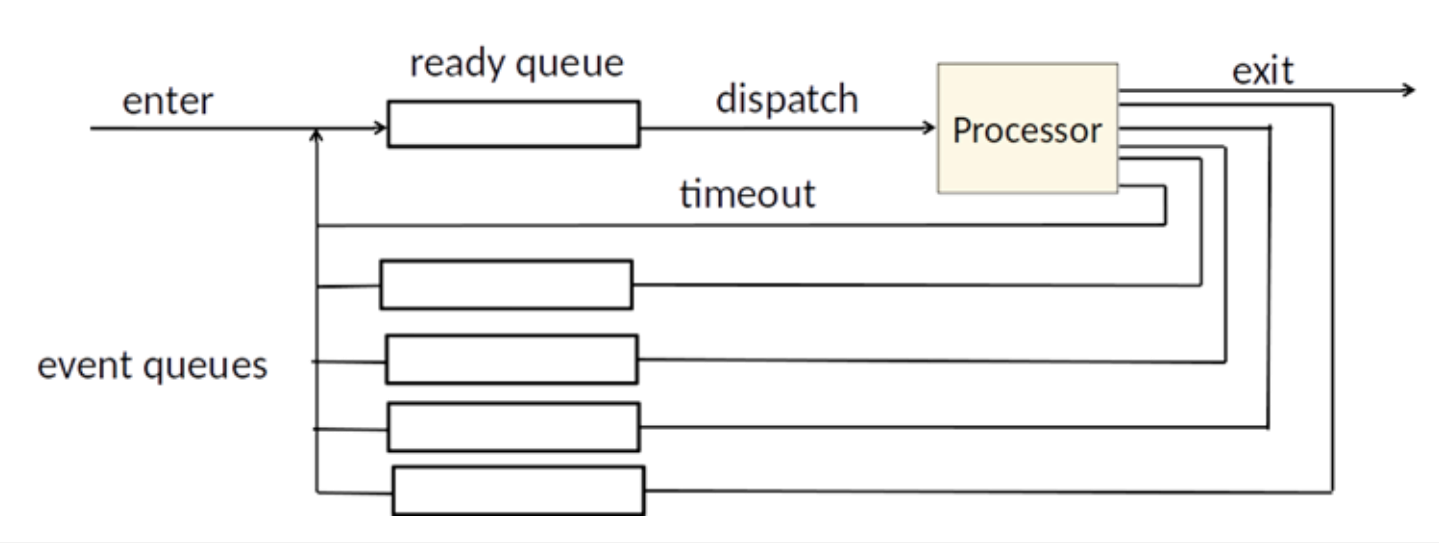
- To reflect the two pause states, use two queues…



Process Management

With single blocked queue, when events occur we need to cycle through the queue to see which process is waiting for the event…

Soluti



Suspended Processes

CPU is faster than I/O, so all processes could be waiting for I/O

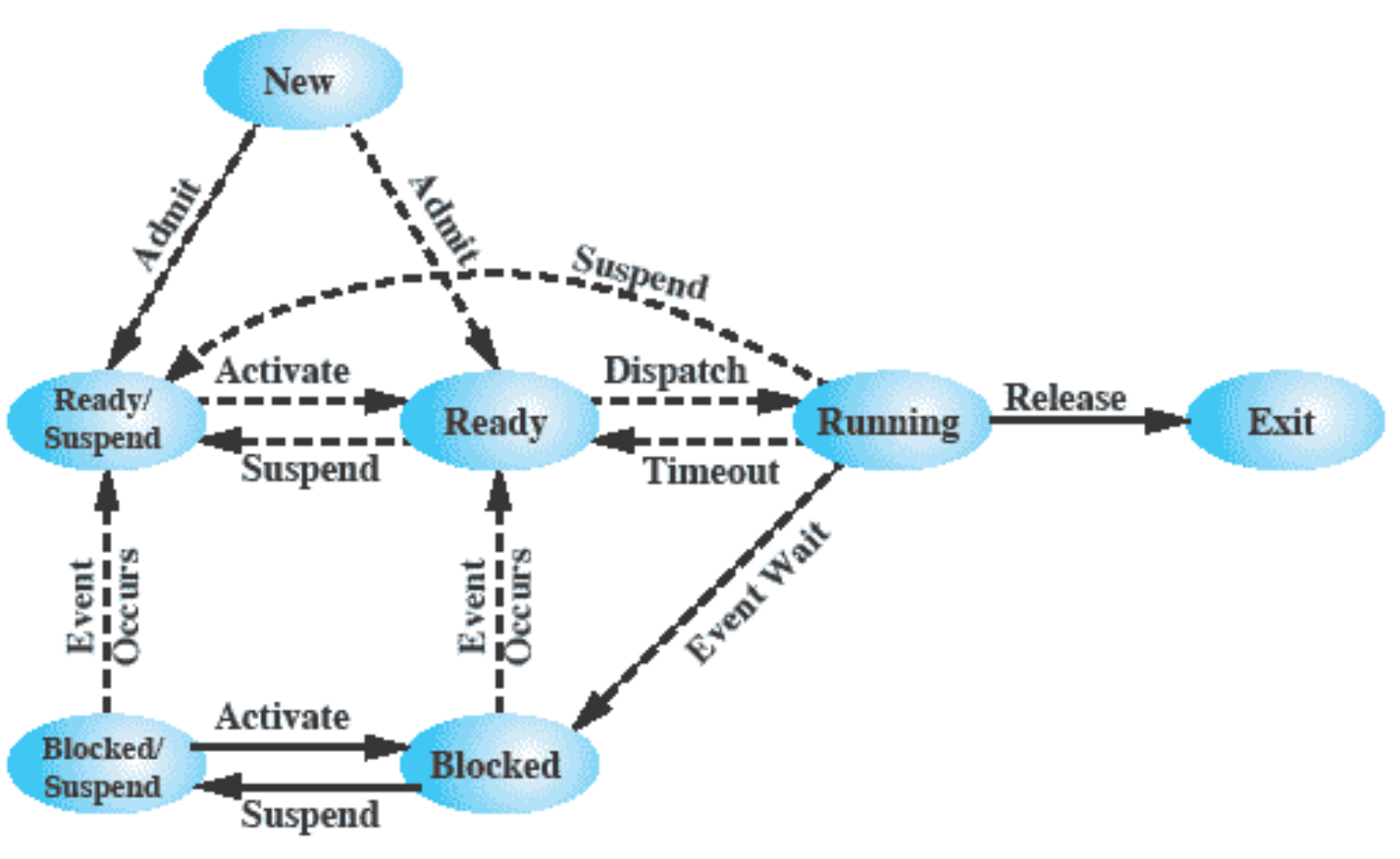
Swap processes to disk to free memory and use processors effectively

Blocked/Waiting becomes **Suspended** when swapped to disk

Two new states:

- **Blocked / Suspend:** Waiting process swapped out
- **Ready / Suspend:** Process could be executed, but swapped out

Suspended Processes



Process Control Block

Data structure used by OS to hold information about a process, including:

- current value of the program counter
- values of all CPU registers for the process
- base and bound register values
- accounting information

- Each state is represented by a list of PCBs, **one for each process** in that state

Process Control Block

- Single CPU implies a set of CPU registers containing values for currently executing process

- Each time a process is moved to the running state:
  - Register values for the currently running process are stored into a PCB
  - That PCB is moved to the list of the state into which it goes
  - Register values of new process moving into the running state are loaded into CPU

- The exchange of register information is called a **context switch**

Goals for Today

- What an operating system is, and where it sits between hardware and applications
- The services it provides, and how it manages resources
- How operating systems evolved: serial processing, batch, multiprogramming, timesharing
- Processes, and the states they move between
- Queues, suspension and swapping to disk
- The process control block, and what happens during a context switch

CS-107 Lecture 15
Operating Systems

Troy Astarte

Megan Venn-Wycherley